

Linux como Servidor

Onde nossa aplicação vai rodar?



"Tá pronto! Funciona no meu PC."

...e o professor pergunta: *qual é o endereço pra eu acessar?*



Seu PC **desliga** à noite



Muda de rede, muda de IP



Ninguém de fora **alcança** ele



Do código ao usuário: a jornada



DIVISÃO DE PAPÉIS

Vocês **constroem** o software.
Aqui, fazemos ele **sair do seu computador** e **chegar até o usuário**.

META DO SEMESTRE

Olhar para <https://projeto.exemplo.com> e explicar **tudo** que acontece, do DNS até o banco de dados.

Ao final desta aula, você vai...

- 1 Acessar um servidor por **SSH** e explicar a conexão: IP, TCP e porta 22
- 2 Descobrir **IP, interface, gateway e rotas** do servidor
- 3 Identificar **processos, portas** e o que **sobe junto** com o servidor
- 4 Diagnosticar um serviço com problema usando **logs** e a **árvore de processos**
- 5 Deixar o servidor **pronto e documentado** para as próximas aulas

"Hoje eu coloquei um serviço no ar num servidor Linux — e entendi por que ele funciona."

O que é um servidor?

SEU COMPUTADOR

- Tem tela, teclado e mouse
- Liga e desliga todo dia
- Usado por **uma** pessoa, na frente dele
- Roda programas **para você**

SERVIDOR

- Normalmente **sem tela** (headless)
- Ligado **24 horas**, com endereço conhecido
- Atende **muitos** clientes, **pela rede**
- Roda **serviços** que ficam esperando pedidos

 Servidor não é um tipo de máquina: é um **papel**. Qualquer computador que **atende pedidos pela rede** está sendo servidor.

Analogia: a cozinha do restaurante 🍴



Cozinha

= servidor

O cliente não entra. Só faz pedidos.



Balcão nº 22

= porta

Cada tipo de pedido chega num balcão.



Cozinheira

= processo

Quem está de fato trabalhando agora.



Escala do dia

= inicialização

Diz quem começa a trabalhar quando a cozinha abre.



Caderno

= logs

Tudo que deu certo e errado fica anotado.

Guarde essa imagem: ela vai aparecer a aula inteira.

Por que Linux? E cadê a tela?



Padrão da nuvem

A maior parte dos servidores web e das máquinas em nuvem roda Linux.



Leve

Sem interface gráfica, sobra CPU e memória para a aplicação.



Livre

Ubuntu Server, Debian, Rocky... gratuitos e com código aberto.

🖥️ Sem interface gráfica, **tudo é feito pelo terminal** — e **pela rede**. Na vida real, quase ninguém "senta na frente" de um servidor.

O cenário do nosso laboratório

SEU PC LINUX • DOCKER ENGINE



Seu PC

cliente • terminal e navegador

172.30.0.1

na rede rede-lab (é o gateway)

SSH → porta 22



HTTP → porta 8080



container srv-turma

Ubuntu 24.04 • faz papel de servidor

172.30.0.10

rede-lab 172.30.0.0/24



Rede da escola

o PC encaminha os pacotes do servidor



Internet

📌 **Esse servidor vai te acompanhar até o Capítulo 9.** Como criar: [preparar-servidor.md](#) • nunca use `docker compose down -v`



Um servidor dentro de um container?

No docker-aula

Um container = **um programa**: nginx, banco, API... É assim que containers devem ser usados.

Neste semestre

O **srv-turma** é uma exceção proposital: tem usuários, SSH, vários processos e logs. Ele **finge ser uma máquina**.

NUMA MÁQUINA FÍSICA OU VM	NO NOSSO CONTAINER	POR QUÊ?
tem o próprio kernel	usa o kernel do seu PC	container isola, não emula hardware
liga com um boot	o processo 1 faz o papel de boot	quando ele termina, o container desliga
<code>sudo reboot</code>	<code>docker compose restart</code>	quem liga e desliga é o Docker
disco guarda tudo	só os volumes sobrevivem à recriação	o resto nasce de novo da imagem

🎯 Todo o resto — SSH, IP, rotas, portas, processos, permissões, logs — funciona **igual a um servidor Linux de verdade**.

SSH: um terminal que viaja pela rede



Cliente

programa **ssh**

172.30.0.1 : **54012**

porta de origem aleatória

conexão TCP criptografada



comandos vão • respostas voltam



Servidor

serviço **sshd**

172.30.0.10 : **22**

porta conhecida, fixa

```
ssh aluno@172.30.0.10
```

programa

o cliente SSH

usuário

conta que existe no servidor

endereço

IP (ou nome) do servidor

Outro caminho: **ssh -p 2222 aluno@localhost** — a porta **2222** do seu PC é redirecionada para a **22** do servidor.



Entrar no servidor

PRÁTICA • ETAPA 1

Terminal do SEU computador

```
ssh aluno@172.30.0.10
Are you sure you want to continue
connecting (yes/no)? yes
aluno@172.30.0.10's password:
# agora você está no servidor
hostname      # nome da máquina
whoami        # qual usuário sou eu?
pwd           # em que diretório estou?
```

✓ FUNCIONOU?

O prompt virou `aluno@ubuntu:~$`. Tudo que você digitar agora roda **no servidor**.

🚨 DEU ERRO?

timed out → o pacote nem chegou. Confira o IP.
refused → chegou, mas ninguém atende na porta 22.

🤔 A pergunta do `yes/no` só aparece na primeira vez. O que o cliente guardou?

Onde fica cada coisa no Linux

```
/      raiz: tudo começa aqui
├── etc/   configurações
├── home/  pastas dos usuários
├── var/log/ logs (o "caderno")
├── srv/   dados dos serviços ← app
├── opt/   programas à parte
└── usr/bin/ comandos: ls, ip...
```

Terminal – servidor

```
cd /
ls
ls /etc | head
ls /var/log
cd ~      # volta pra home
pwd
```

 No Linux não existe **C:** nem **D:**. Tudo — inclusive discos e pendrives — fica pendurado em algum lugar abaixo de **/**.

Usuários e permissões

TEORIA + PRÁTICA

```
-rw-r--r-- 1 root root 7 set 16 /etc/hostname
```

rw- dono lê e escreve

r-- grupo só lê

r-- outros só leem

root root dono e grupo

Terminal – provoque o erro

```
ls -l /etc/hostname
```

```
touch /etc/teste
```

```
touch: cannot touch '/etc/teste':
```

```
Permission denied
```

```
sudo touch /etc/teste # e agora?
```

```
ls -l /etc/teste
```

```
sudo rm /etc/teste
```

root = administrador, pode tudo.

sudo = "rode este comando como root".

🤔 Se um invasor entrar pela aplicação, o que ele consegue fazer se ela roda como **root**? E como **www-data**?

Como o servidor se conecta à rede



CONCEITO	EM UMA FRASE
Interface	Uma "placa de rede", física ou virtual. Cada uma tem seu IP.
IP / máscara	/24 = os primeiros 3 números são a rede: 172.30.0.x
Gateway	A saída da rede local. Destino fora da rede? Manda pra ele.
Rota default	A regra "tudo que não é daqui vai pelo gateway".

Conceito → comando

QUERO SABER...	COMANDO	O QUE PROCURAR NA SAÍDA
Meu IP e interface	<code>ip addr</code>	linha inet da interface que não é lo
Gateway e rotas	<code>ip route</code>	default via ...
Chego lá?	<code>ping -c 4 destino</code>	0% packet loss e o tempo em ms
O site responde?	<code>curl url</code>	o conteúdo (ou o erro) devolvido
O que está rodando	<code>ps aux · top</code>	comando, PID , usuário, %CPU, %MEM
Quem criou o processo	<code>pstree -p</code>	o processo pai de cada um
Memória e disco	<code>free -h · df -h</code>	available · Use%
Portas abertas	<code>sudo ss -tulpn</code>	LISTEN , endereço:porta, processo
O que sobe no boot	<code>ls -l /etc/inicio.d</code>	quais arquivos são executáveis (x)
O que aconteceu	<code>sudo tail /var/log/X.log</code>	mensagens de erro e horário

Não decore comandos: pergunte o que você quer saber e escolha a ferramenta.



Qual é o endereço do servidor?

Terminal – servidor

```
ip addr
```

```
1: lo: <LOOPBACK,UP,LOWER_UP> mtu 65536 qdisc noqueue state UNKNOWN
    inet 127.0.0.1/8 scope host lo
11: eth0@if38: <BROADCAST,MULTICAST,UP,LOWER_UP> mtu 1500 qdisc noqueue state UP
    link/ether 4e:5e:28:dd:23:21 brd ff:ff:ff:ff:ff:ff link-netnsid 0
    inet 172.30.0.10/24 brd 172.30.0.255 scope global eth0
```

① **eth0** — nome da interface numa VM seria enp0s3, ens3...

② **172.30.0.10/24** — IP e tamanho da rede

③ **4e:5e:28:...** — MAC, o "RG" da placa

🔄 Agora, no seu PC: `ip addr`. Ache a interface `br-`... com **172.30.0.1** e rode `ping 172.30.0.10`.

🤔 Um colega, de outro PC, consegue chegar em 172.30.0.10? Então para que serve a porta 2222?

Por onde os pacotes saem?

Terminal – servidor

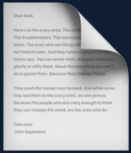
```
ip route
default via 172.30.0.1 dev eth0
172.30.0.0/24 dev eth0 proto kernel scope link src 172.30.0.10
```

Diagnóstico em camadas: rode na ordem e anote o resultado

TESTE	SE FUNCIONAR, PROVA QUE...
<code>ping -c 4 172.30.0.1</code>	chego ao gateway (rede local ok)
<code>ping -c 4 8.8.8.8</code>	saio para a internet (rota ok)
<code>ping -c 4 github.com</code>	nomes viram IP (DNS ok) → cap. 5
<code>curl -I https://github.com</code>	um serviço web responde (HTTP ok)

🤔 O ping para **8.8.8.8** funciona, mas para **github.com** não. Onde está o problema? O que **não** é o problema?

Programa, processo, serviço e porta



Programa

Arquivo parado no disco.

`/usr/bin/python3`

a receita



Processo

Programa em execução. Tem PID, pai, usuário, CPU e RAM.

`PID 17 · PPID 16`

a cozinheira trabalhando



Serviço

Processo que roda em segundo plano, esperando pedidos, e **sobe no boot**.

`sshd`

contratada, com turno fixo



Porta

Número onde o serviço espera conexões.

`22 · 80 · 443 · 8080`

o balcão numerado

🔑 Um endereço completo de serviço é **IP + porta**: `172.30.0.10:22`. O IP acha a máquina; a porta acha o serviço.

O servidor está saudável?

Terminal – servidor

```
nproc          # núcleos de CPU
4

free -h        # memória
              total  used  free  available
Mem:   15Gi  3.1Gi  8.2Gi   11Gi

df -h /        # disco
Filesystem    Size  Used Avail Use%
overlay       234G   61G  162G   28%

uname -r       # versão do kernel

ps aux        # quantos processos?

top           # aperte q para sair
```

OLHE PARA...

ALERTA

%CPU no top

100% o tempo todo

available

perto de zero

Use%

acima de 90%



Rode `nproc`, `free -h` e `uname -r` também **no seu PC**. Por que os números são iguais? E o `ps aux`, por que é tão diferente?

Quem está escutando?

Terminal – servidor

```
sudo ss -tulpn
```

Netid	State	Local Address:Port	Peer Address:Port	Process
udp	UNCONN	127.0.0.11:59537	0.0.0.0:*	
tcp	LISTEN	0.0.0.0:22	0.0.0.0:*	users:(("sshd",pid=7,fd=4))
tcp	LISTEN	127.0.0.11:40345	0.0.0.0:*	
tcp	LISTEN	:::22	:::*	users:(("sshd",pid=7,fd=5))

Decifrando -tulpn

-t TCP -u UDP -l só quem escuta (LISTEN)
-p mostra o processo -n números, sem traduzir
nomes
sem sudo, a coluna Process vem vazia

Endereço de escuta

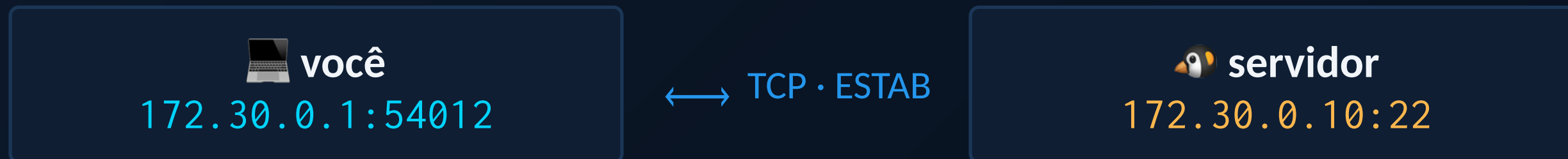
0.0.0.0:22 → aceita conexões vindas de **qualquer rede**
127.0.0.11 → só a **própria máquina** acessa
é o DNS interno do Docker → cap. 3 · [::] = 0.0.0.0 em
IPv6

Encontre a sua própria conexão

```
Terminal - servidor

ss -tn
State Recv-Q Send-Q Local Address:Port Peer Address:Port
ESTAB 0 0 172.30.0.10:22 172.30.0.1:54012

pstree -p
```



👁️ No `pstree -p`, ache o `sshd` da sua sessão e o `pstree` que você acabou de rodar.

🤔 Por que do lado do cliente a porta é **54012** e não 22?

O que sobe junto e onde fica anotado

Terminal – servidor

```
ls -l /etc/inicio.d           # o que o "boot" deste servidor executa
sudo cat /var/log/boot.log    # o que aconteceu quando ele ligou
sudo tail -5 /var/log/auth.log # logins pelo SSH
Accepted password for aluno from 172.30.0.1 port 54012 ssh2
```

QUERO...

NESTE SERVIDOR

ligar **agora**

executar o script

desligar **agora**

`kill` no processo certo

ligar / não ligar **no boot**

`chmod +x` · `chmod -x`

 Ache no log o **seu** login. IP e porta batem com o `ss -tn`?

 Em VMs e servidores reais quem faz isso é o **systemd**: `systemctl start|enable` e `journalctl`. A ideia é a mesma.

Missão: o servidor herdado

PRÁTICA • ETAPA 5

A equipe anterior deixou este servidor configurado com a **app-turma** na porta **8080**. Antes de ir embora, jurou que "estava tudo funcionando".

Terminal – servidor

```
# 1. baixar o laboratório
curl -O https://raw.githubusercontent.com/eversonscherrer/\
infra-devops-aula/HEAD/capitulo-01/lab01.sh

# 2. receber o servidor no estado em que foi entregue
sudo bash lab01.sh preparar
```

RELATO DO CLIENTE

"Quando eu acesso a porta 8080 aparece uma página estranha, não é a nossa aplicação..."

REGRA DO JOGO

Não abra o **lab01.sh** atrás da resposta.
Investigue como profissional: **com os comandos da aula.**

Sintoma: página errada na porta 8080

Terminal – servidor

```
curl http://localhost:8080
```

...

```
<h1>Painel Administrativo v0.3</h1>
```

```
<p>Sistema em manutenção desde 2019.</p>
```

...

⚠ Pensou em **kill**? Pode tentar. Depois espere **5 segundos** e rode o **curl** de novo. **Por que** isso aconteceu?

Roteiro de investigação

Responda cada pergunta **antes** de corrigir qualquer coisa.

- 1 A **app-turma** está rodando? → `ps aux · pgrep -a`
- 2 O que ela registrou quando tentou subir? → `/var/log`
- 3 Quem está ocupando a porta 8080? → `ss`
- 4 Quem é o **pai** desse processo? Como ele subiu? → `ps tree -p · /etc/inicio.d`
- 5 Como tirar o intruso **de vez** e subir a app-turma?

Pistas para interpretar o que você vê

VOCÊ VIU...

SIGNIFICA QUE...

`pgrep` não devolve nada

não existe processo com esse nome: o serviço **não está rodando**

`Address already in use`

outro processo **já ocupa** aquela porta

`vigia.sh(14)-runuser(16)-python3(17)`

cada processo foi **criado** pelo que está à sua esquerda (o **pai**)

`while true ... sleep 3`

um laço que **religa** o programa sempre que ele morre

`-rwxr-xr-x` em `/etc/inicio.d`

executável: **sobe no boot** · sem **x**: não sobe

`run-parts: executing ...`

no `boot.log`: o que foi ligado e **em que ordem**

✓ ACHA QUE RESOLVEU?

```
sudo bash lab01.sh verificar
```

As 5 primeiras linhas são a **missão**. As outras 3 são o **desafio final** →

Entendeu ou só copiou?

- 1 Por que **kill** no python não resolveu o problema de vez?
- 2 Se você só matasse o **vigia**, o que aconteceria no próximo boot?
- 3 A app escuta em **0.0.0.0:8080**. E se fosse **127.0.0.1:8080**, quem conseguiria acessar?
- 4 Por que dois processos não podem escutar na **mesma porta** do mesmo IP?
- 5 No **/var/log/app-turma.log**, de qual IP veio o acesso do seu navegador?
- 6 Por que a app-turma roda como **www-data** e não como root?

As respostas vão para a ficha do servidor e para o AVA.

Deixe o servidor pronto

DESAFIO FINAL



Renomeie

Hostname no padrão `srv-grupo3`

pista: quem define o nome é o `compose.yaml`



Identifique

Troque **SEU GRUPO AQUI** em

`/srv/app-turma/index.html`

pista: `ls -l` • `sudo nano`



Acesse de fora

Abra a app pelo **IP do servidor** e por `localhost:8080`. Um colega consegue abrir pelo **IP do seu PC**?



Sobreviva ao boot

`docker compose restart` no PC, reconecte.

Continua tudo no ar?

pista: o que sobe sozinho?

🏁 META

`sudo bash lab01.sh verificar` → 8 de 8



Para trocar o hostname o container foi **recriado**. Por que a página editada e as correções da missão continuaram lá?



O que entregar no AVA



Ficha do servidor

`ficha-servidor.md`
preenchida: IP, gateway,
recursos, portas, processos e
respostas.



Saída do verificador

O bloco **Evidência** do
`lab01.sh` `verificar`, com 8
de 8.



Uma frase

"Hoje eu consegui ___
e entendi que ___."

 O servidor fica guardado nos **volumes**. Pode desligar com `docker compose stop` — mas **nunca** use `down -v`. Na próxima aula partimos daqui.

PERGUNTA RÁPIDA

O `sudo ss -tulpn` mostra `tcp LISTEN 127.0.0.1:5432`.

Um colega tenta acessar essa porta de outro PC, pelo IP do servidor. O que acontece?

- A Conecta, porque a porta está em LISTEN
- B Não conecta: o serviço só aceita conexões da própria máquina
- C Conecta, desde que o colega use sudo
- D Conecta, mas só pela porta 22



Resumo: sua caixa de ferramentas

Acesso

```
ssh usuario@ip  
hostname · whoami
```

Arquivos

```
pwd · cd · ls -l  
sudo <comando>
```

Rede

```
ip addr · ip route  
ping -c 4 · curl
```

Recursos

```
ps aux · top · pstree -p  
free -h · df -h · nproc
```

Portas

```
sudo ss -tulpn  
ss -tn
```

Boot e logs

```
ls -l /etc/inicio.d  
chmod +x | -x · kill  
sudo tail /var/log/*.log
```

🧠 Sequência de diagnóstico que vale para o semestre todo: **o serviço está rodando? → o que dizem os logs? → está escutando na porta certa? → a rede chega até ele?**



Servidor pronto... mas vazio.

Como o código que a equipe escreve chega até ele?



Próxima aula — Capítulo 2: Git e GitHub na infraestrutura

Não apague os volumes do srv-turma. Ele é o nosso servidor até o fim do semestre.